Title

Library Management System with Borrow/Return Tracking

Problem Statement

Libraries often manage a large number of books and borrowers. Manual tracking of issued and returned books can lead to errors such as lost records, duplicate entries, or incorrect availability status.

The **Library Management System** aims to automate book management by allowing users to add books, borrow books, return books, and track availability. The system ensures that a book cannot be issued if it is already borrowed and updates availability automatically.

Target User

Students and librarians in educational institutions.

Core Features

1. Add new books to the library
2. Display all available books
3. Borrow a book
4. Return a book
5. Track issued books
6. Prevent borrowing of unavailable books
7. Handle invalid input using exception handling

OOP Concepts Used

Abstraction
Abstract class `LibraryItem` defines common attributes like ID and title.

Inheritance
`Book` class extends `LibraryItem`.

Encapsulation
Private variables with getters/setters.

Polymorphism
Overridden `display()` method.

Collections
`ArrayList` used to store books.

Exception Handling
Handles invalid input and unavailable books.

4. Return a book
5. Track issued books
6. Prevent borrowing of unavailable books
7. Handle invalid input using exception handling

OOP Concepts Used

Abstraction
Abstract class `LibraryItem` defines common attributes like ID and title.

Inheritance
`Book` class extends `LibraryItem`.

Encapsulation
Private variables with getters/setters.

Polymorphism
Overridden `display()` method.

Collections
`ArrayList` used to store books.

Exception Handling
Handles invalid input and unavailable books.

System Architecture

```
LibraryItem (Abstract Class)
        ↓
      Book Class
        ↓
```

```
  Library Manager
        ↓
    Main Program
```

Java Code (Menu Driven)

```
import java.util.*;

// Abstract class
abstract class LibraryItem {
    protected int id;
    protected String title;

    LibraryItem(int id, String title) {
        this.id = id;
        this.title = title;
    }

    abstract void display();
}

// Book class
class Book extends LibraryItem {
    private boolean issued;

    Book(int id, String title) {
        super(id, title);
        issued = false;
    }

    public boolean isIssued() {
        return issued;
    }

    public void borrowBook() {
        issued = true;
    }
```

```
    public void returnBook() {
        issued = false;
    }

    @Override
    void display() {
        System.out.println("Book ID: " + id +
                " | Title: " + title +
                " | Status: " + (issued ? "Issued" : "Available"));
    }
}

// Library Manager
class LibraryManager {
    private List<Book> books = new ArrayList<>();

    public void addBook(Book b) {
        books.add(b);
        System.out.println("Book added successfully.");
    }

    public void showBooks() {
        if (books.isEmpty()) {
            System.out.println("No books available.");
            return;
        }

        for (Book b : books) {
            b.display();
        }
    }

    public void borrowBook(int id) {
        for (Book b : books) {
            if (b.id == id) {
                if (!b.isIssued()) {
                    b.borrowBook();
                    System.out.println("Book borrowed successfully.");
                } else {
                    System.out.println("Book already issued.");
                }
                return;
```

```
            }
        }
        System.out.println("Book not found.");
    }

    public void returnBook(int id) {
        for (Book b : books) {
            if (b.id == id) {
                if (b.isIssued()) {
                    b.returnBook();
                    System.out.println("Book returned successfully.");
                } else {
                    System.out.println("Book was not issued.");
                }
                return;
            }
        }
        System.out.println("Book not found.");
    }
}

// Main class
public class LibrarySystem {
```

```
            }
        }
        System.out.println("Book not found.");
    }

    public void returnBook(int id) {
        for (Book b : books) {
            if (b.id == id) {
                if (b.isIssued()) {
                    b.returnBook();
                    System.out.println("Book returned successfully.");
                } else {
                    System.out.println("Book was not issued.");
                }
                return;
            }
        }
        System.out.println("Book not found.");
    }
}

// Main class
public class LibrarySystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        LibraryManager manager = new LibraryManager();
        int choice;

        do {
            try {
                System.out.println("\n--- Library Management System ---");
                System.out.println("1. Add Book");
                System.out.println("2. Show Books");
                System.out.println("3. Borrow Book");
                System.out.println("4. Return Book");
                System.out.println("0. Exit");

                System.out.print("Enter choice: ");
                choice = sc.nextInt();

                switch (choice) {
```

```
                case 1:
                    System.out.print("Enter Book ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Book Title: ");
                    String title = sc.nextLine();

                    manager.addBook(new Book(id, title));
                    break;

                case 2:
                    manager.showBooks();
                    break;

                case 3:
                    System.out.print("Enter Book ID to borrow: ");
                    manager.borrowBook(sc.nextInt());
                    break;

                case 4:
```

```
                case 1:
                    System.out.print("Enter Book ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Book Title: ");
                    String title = sc.nextLine();

                    manager.addBook(new Book(id, title));
                    break;

                case 2:
                    manager.showBooks();
                    break;

                case 3:
                    System.out.print("Enter Book ID to borrow: ");
                    manager.borrowBook(sc.nextInt());
                    break;

                case 4:
                    System.out.print("Enter Book ID to return: ");
                    manager.returnBook(sc.nextInt());
                    break;

                case 0:
                    System.out.println("Exiting program...");
                    break;

                default:
                    System.out.println("Invalid choice.");
            }

        } catch (Exception e) {
            System.out.println("Invalid input. Try again.");
            sc.nextLine();
            choice = -1;
        }

    } while (choice != 0);

    sc.close();
```

```
    }
}
```